

Making Every Experiment Count

A cheap guide for expensive tests

Hello everyone, I am **Wenje Ma**. Today I want to tell you about a piece of work I did. It all comes down to **one question**: when every experiment is expensive and you can only afford a few, how do you make **each one count**? Here's my plan for the next fifteen minutes. First, a quick story to set up the problem. Then the two simple ideas my method is built on. Then what I found when I tested it. And finally, where I'd like to go next.

Begin: A Story

Let me start with the story. Imagine you work for a company, and your job is to **tune parameters**. You have an experiment — an engine, a material, a production process — but every run costs money — a real physical test, a real engine, a real batch of material — and you can only afford **fifteen runs** in total. Here's the hard part: more than ten parameters, and **they affect each other**. Change one, the others move too — like an engine: raise the pressure, and the temperature and the fuel mix all shift together. So you can't tune them one at a time. You need the **best combination of all at once**, and you get only fifteen chances to find it. What would you do? That is exactly the problem I worked on. And it's not just a story — engines, materials, biology: many real systems are exactly like this. Testing is expensive, few chances. So the real question is, how do you **spend a tiny budget smartly**?

Background: Black Box and Budget

Why is this hard? Two reasons. First, the **black box**. A black box means you only see input and output, never the inside. It's like a **locked machine** — you put in a coin and read the result, but you can't open it. Our function is exactly that: we don't know its shape, we can only poke it and watch. In real life that black box is usually a simulation or a test bench — expensive to run, slow to answer, and we never see the internal equations. Second, the **budget**. We can run the real experiment only fifteen times, but the space of settings is enormous. With that many parameters and that few chances, **guessing is hopeless**. So here's the

problem I set myself: use fifteen runs to find the **best point** of a high-dimensional black box.

The Two Channels

Now the good news: our black box has **two channels**. One is cheap and fast — we can run it hundreds of times — but it has **bias**, meaning it isn't fully accurate. It's like a rough map: the shape is there, the details are off. The other is expensive and accurate, and we can only afford it fifteen times. So the question becomes: how do we use the cheap channel to **guide the expensive one**? My idea is simple. Think of the accurate answer as the cheap answer **plus a correction** — a bias:

$$h_k(\mathbf{x}) = h_{k-1}(\mathbf{x}) + \delta_k(\mathbf{x})$$

That's the whole idea in one line: the accurate function is the cheap function plus a correction. If we can learn that correction from a few accurate runs, all those cheap runs suddenly become useful for finding the minimum. It's like having a cheap friend who can run anywhere in the city quickly but only guesses the address, and an expensive friend who goes straight there but can only do it a few times. We let the **cheap friend scout**, and the **expensive friend confirm**.

Where the Pieces Come From

Almost the whole skeleton of this method comes from **one person** — V. Roshan Joseph, a big name in experimental design. Let me show his picture:



He gave us **three pieces**, and they map onto three questions. First, where do we put our first experiments? That's the **maxpro design**. Second, how do we add experiments one at a time? That's the **sequential design**. Third, how do we combine cheap and expensive data? That's the **fusion step**. He built each piece separately, but never joined them end to end. So my work is simpler than it sounds: I took his three pieces, connected them into **one pipeline**, and tested whether each piece earns its place.

Piece One: Where to Start

First piece — where to start. At the very beginning we know nothing about the function, so the first points should **cover the space evenly** — not piled in one

corner, not overlapping. Maxpro design does exactly that. The name, **maximum projection**, means it keeps points spread out not just in the whole space, but in every low-dimensional shadow — every projection. So even if two points look close when you squash the space down, they're still far apart in the full space. The criterion is this equation, and the effect is in the figure:

$$\min_D \sum_{i=1}^n \prod_{j \neq i} \frac{1}{\sum_{k=1}^p (x_{ik} - x_{jk})^2}$$



Piece Two: The Stand-In Model

The second piece is the model that **guesses the shape** of the black box. That stand-in model is called a **Gaussian process**. Here's the simplest way to think about it. A normal distribution describes one number — it gives you the average and how spread out it is. A Gaussian process is the same idea, but for a **whole function** instead of one number. So instead of "the answer is around here," it gives a whole curve — and, just as important, it tells you **how unsure** it is at every point along that curve. That uncertainty is what makes it a guide rather than a guesser — a smart guesser that also tells you how much to trust each guess. We need it, because we have to decide where to test next — and the best place to test is where we're **hopeful and uncertain**.

The Whole Pipeline

Now the whole pipeline, in **four steps**. One, choose the starting points, spread out evenly. Two, feed the model with many cheap points; they go into the Gaussian process, and it draws a **rough map**. Three, run **one expensive test** where the model says the valley bottom probably is. Four, repeat — every round the map gets sharper, and we get closer to the best point. It's exactly how a human engineer would work: map, guess, test, refine.

Our Question

Here is the **formal question** I want to answer:

$$\hat{\mathbf{x}} \approx \arg \min_{\mathbf{x} \in [0,1]^p} h_{\mathbf{H}}(\mathbf{x})$$

Find the point that makes the expensive function as small as possible, using only fifteen precise tests. I broke this into **three smaller questions**. First, does combining the two channels actually help? Second, does **screening** help — using cheap tests to decide which parameters matter, and dropping the rest? And third, does the whole thing still work **in high dimensions**?

How I Measure Success

Before I show results, let me say how I measure success, because it matters. I ran the whole pipeline **twenty times** from scratch, because the starting points are random. Each run gives me the best value it found. Then I take the **median** of those twenty best values — the middle one — as the number to report. Twenty runs is our way of being honest: one lucky run proves nothing; we want what happens on average. A lower number means a better point. So in every figure, **lower is better**.

Enough setup. Let me show you **what actually happened** when I ran it.

Result 1: Combination Is the Key



First result: **combination is the key**. With the same budget, a few precise plus many cheap beats only precise. Low dimensions are the clearest. In one dimension, precise tests alone can't find the best point — it stays at zero, meaning it never improved. Add the cheap tests and the fusion step, and it **jumps to the true best point**. In two and four dimensions, it lands very close to the known best answer. So if your tests are expensive, the message is clear: don't just run the expensive one — **bring a cheap one along to help**.

Result 2: Screening Is Useless



Second result: **screening is useless** — it even hurts. Screening ranks the parameters with cheap tests, and keeps only the ones it calls important. But the cheap tests are **biased** — so the ranking is unreliable: the parameter it calls important might not matter at all in the accurate function. When I removed the screening step, the results actually got better. My honest answer to that question is

no: screening doesn't help, at least the way I set it up. It's a nice **counterintuitive finding**: the step that sounds like it saves work actually costs you accuracy.

Result 3: High Dimensions Fail



Third result: **high dimensions fail**. In eight dimensions, even fusion stops working — the **curse of dimensionality**. Here's what that phrase means. To sketch an 8-D landscape, you need points scattered through eight dimensions; but we have only fifteen precise points — far too few to cover a space that big. It's the price of exploring a space that's too big for the points we can afford — the classic story of high dimensions: more space, the same few points. The map stays nearly empty, and the method gets lost. In low dimensions, fusion is the difference between failing and finding the optimum. In high dimensions, even fusion has a **ceiling**.

The Whole Picture



Put all the results in one figure and the picture is clear. The **complete pipeline** is best in low dimensions. Drop the fusion step, and it breaks even in low dimensions. In high dimensions, nothing works. So it all comes down to **one sentence**: it's not about spending more money on precise tests — it's about **making each precise test more valuable**. And the limit is set by how many dimensions you're working in — the story just gets worse the higher you go.

Future Directions

Three directions next. First, **when is screening trustworthy**? It failed because of bias. So the open question is: under what conditions does the cheap-test ranking actually match the true ranking? If we know that, we can check before screening, and only screen when it's safe. Second, how do we **break the curse of dimensionality**? It already fails at eight dimensions. Maybe we can put some structure on the bias, or shrink the dimension before we start. The first step is probably just to push to more dimensions and find where the line is. Third, **push it into the real world** — engines, materials, biology, anywhere experiments

actually cost money. Maybe even build a **tool**: you tell it your budget, and it gives you the best plan on its own.

Summary

Let me close with one sentence. When experiments are expensive, the smartest thing is not to run more of them — it's to **make every single one count**. That's what I set out to do — a cheap guide to a few expensive tests. Thank you for listening!